

Final Project Proposal and Outline

Course Number CST110/299 – Introduction to Cloud Computing

Professor Jason Wright

7/1/26

My cloud architecture will be solving a three part business problem involving infrastructure, automation, monitoring, and a continuous integration and continuous deployment (CI/CD) pipeline. A small company needs a secure, repeatable cloud environment they can deploy consistently across development, staging, and production without manual configuration steps that introduce errors. This company cannot afford to have an engineer manually checking their systems around the clock. The project will automate the monitoring so the engineer only gets involved when something needs attention in order to catch problems before they become outages. Finally, each change to cloud infrastructure will go through a controlled auditable process that prevents unauthorized or untested changes from reaching production.

I will be using Terraform and GitHub in addition to several AWS services for this project. Terraform serves as the Infrastructure as Code (IaC) platform used to provision and manage all AWS resources. Every component will be defined in reusable configuration files to create a repeatable deployment process that eliminates manual configuration errors and provides consistency. GitHub will serve as the code repository for Terraform configurations and project documentation, as well as providing version control.

Amazon Virtual Private Cloud (VPC) will provide the isolated network environment that hosts all cloud resources. Amazon Elastic Compute Cloud (EC2) will provide the virtual machine that will host the application. It will also serve as the monitored resource throughout the automation and monitoring portion of the project. Amazon Simple Storage Service (S3) will be used as object storage for Terraform state files, application logs, backups, and deployment artifacts. AWS identity and Access Management (IAM) will manage authentication and authorization for users, services, and applications, enforcing least-privilege access. Amazon

CloudWatch will provide centralized monitoring for infrastructure health and performance.

Amazon Simple Notification Service (SNS) will deliver automated notifications when CloudWatch alarms are triggered. AWS Systems Manager will enable secure administration of EC2 instances without requiring SSH access. AWS CodePipeline will orchestrate the CI/CD workflow. AWS CodeBuild will perform automated validation of the Terraform code during each pipeline execution. Amazon DynamoDB will be used to provide state locking for the Terraform remote backend, this will prevent multiple deployments from modifying the same infrastructure simultaneously. Optionally AWS Lambda may be used to execute automated actions in response to CloudWatch alarms or SNS notifications.

One pillar of the Well-Architected Framework that will be my primary focus will be Security. One of the business problems I'm solving is to deploy a secure cloud environment consistently across development, staging, and production. I will be using security groups to enforce least-privilege access, IAM roles and policies to restrict permissions to only what is required, EC2 instances within private subnets to minimize exposure, and AWS Systems Manager instead of SSH for secure administration access.

The other Well-Architected Framework that will be my primary focus will be Operational Excellence. Another business problem I will be solving is to automate infrastructure deployment, monitoring, and change management. Terraform will allow for repeatable infrastructure provisioning, eliminating manual deployment errors. CloudWatch will continuously monitor system health, collect logs, and trigger alarms when predefined thresholds are exceeded. Simple Notification Service will then notify administrators when intervention is required, reducing the need for constant monitoring. CodePipeline and CodeBuild will automate validation, testing, approval, and deployment of infrastructure changes.